

Pandas Cheatsheet

Essential syntax for Machine Learning beginners

Basics & Setup

<code>import pandas as pd</code>	Standard import alias
<code>import numpy as np</code>	NumPy companion for numeric ops
<code>pd.DataFrame({'a':[1,2], 'b':[3,4]})</code>	Create DataFrame from dict
<code>pd.Series([1,2,3], name='col')</code>	Create a Series (single column)
<code>pd.set_option('display.max_columns', None)</code>	Show all columns in output

Load & Save Data

<code>pd.read_csv('file.csv')</code>	Load CSV — most common ML format
<code>pd.read_csv('file.csv', index_col=0)</code>	Use first column as index
<code>pd.read_csv('file.csv', usecols=['a','b'])</code>	Load only specific columns
<code>pd.read_csv('file.csv', nrows=1000)</code>	Load first 1000 rows only
<code>pd.read_excel('file.xlsx')</code>	Load Excel file
<code>df.to_csv('out.csv', index=False)</code>	Save to CSV (skip writing index)
<code>df.to_parquet('out.parquet')</code>	Fast binary format — best for ML data

Explore & Inspect

<code>df.head(5)</code>	First 5 rows
<code>df.tail(5)</code>	Last 5 rows
<code>df.sample(5)</code>	5 random rows
<code>df.shape</code>	Returns (rows, columns) tuple
<code>df.dtypes</code>	Data type of each column
<code>df.info()</code>	Column types + non-null counts — run first!
<code>df.describe()</code>	Stats: mean, std, min, max for numeric cols
<code>df.isnull().sum()</code>	Count missing values per column
<code>df.isnull().mean() * 100</code>	% missing per column
<code>df['col'].value_counts()</code>	Frequency of each unique value
<code>df['col'].nunique()</code>	Count distinct values

Select & Filter

<code>df['col']</code>	Select single column → Series
<code>df[['col1', 'col2']]</code>	Select multiple columns → DataFrame
<code>df.iloc[0]</code>	First row by integer position

<code>df.iloc[0:5]</code>	First 5 rows by position
<code>df.loc[3, 'col']</code>	Row index 3, named column
<code>df[df['age'] > 30]</code>	Filter rows where age > 30
<code>df[df['city'] == 'Delhi']</code>	Filter rows matching a value
<code>df[(df['a']>1) & (df['b']<5)]</code>	Multiple conditions — use & not 'and'
<code>df[df['col'].isin(['a', 'b'])]</code>	Value is in a list
<code>df.query('age > 30 and score < 90')</code>	Query syntax — readable for complex filters

Clean Data	
<code>df.dropna()</code>	Drop rows with any NaN
<code>df.dropna(subset=['col1', 'col2'])</code>	Drop rows where specific cols are NaN
<code>df.fillna(0)</code>	Fill all NaN with 0
<code>df['col'].fillna(df['col'].mean())</code>	Fill NaN with column mean — common in ML
<code>df['col'].fillna(method='ffill')</code>	Forward fill — use previous value
<code>df.drop_duplicates()</code>	Remove duplicate rows
<code>df['col'].astype(int)</code>	Convert column to int
<code>df['col'].astype('category')</code>	Category type — saves memory for ML
<code>pd.to_numeric(df['col'], errors='coerce')</code>	Convert to number; errors become NaN
<code>pd.to_datetime(df['col'])</code>	Parse strings to datetime
<code>df.rename(columns={'old': 'new'})</code>	Rename columns
<code>df.drop(['c1', 'c2'], axis=1)</code>	Drop columns (axis=1)

Transform & Mutate	
<code>df['new'] = df['a'] + df['b']</code>	Add computed column
<code>df['new'] = df['col'].apply(func)</code>	Apply a Python function to every value
<code>df['new'] = df['col'].map({'a':1, 'b':2})</code>	Map values using a dictionary
<code>df.assign(new=df['a']*2)</code>	Create column — chainable, returns copy
<code>df['col'] = df['col'].str.lower()</code>	Lowercase string column
<code>df['col'] = df['col'].str.strip()</code>	Remove leading/trailing whitespace
<code>df.sort_values('col', ascending=False)</code>	Sort descending
<code>df.reset_index(drop=True)</code>	Reset to 0,1,2... index
<code>df['col'].clip(0, 100)</code>	Clip values to range [0, 100]

Group & Aggregate	
<code>df.groupby('col').mean()</code>	Mean of all numeric cols per group

<code>df.groupby('col')['val'].sum()</code>	Sum of one column per group
<code>df.groupby('col').agg({'a': 'mean', 'b': 'sum'})</code>	Different aggregations per column
<code>df.groupby('col').agg(['mean', 'std'])</code>	Multiple stats for each group
<code>df.groupby(['c1', 'c2']).size()</code>	Count rows per combination
<code>df.groupby('col').transform('mean')</code>	Group mean, same shape as original
<code>df.pivot_table(values='v', index='r', columns='c')</code>	Excel-style pivot table

Merge & Concatenate

<code>pd.concat([df1, df2])</code>	Stack DataFrames vertically
<code>pd.concat([df1, df2], axis=1)</code>	Join side by side
<code>pd.merge(df1, df2, on='key')</code>	Inner join on key column
<code>pd.merge(df1, df2, on='key', how='left')</code>	Left join — keep all df1 rows
<code>pd.merge(df1, df2, on='key', how='outer')</code>	Outer join — keep all rows
<code>pd.merge(df1, df2, left_on='a', right_on='b')</code>	Join on different column names

ML Preprocessing

<code>x = df.drop('target', axis=1)</code>	Features matrix
<code>y = df['target']</code>	Target variable
<code>pd.get_dummies(df, columns=['cat'])</code>	One-hot encode categorical columns
<code>pd.get_dummies(df, drop_first=True)</code>	One-hot; drop first to avoid multicollinearity
<code>df['col'].astype('category').cat.codes</code>	Label encode — categories to integers
<code>df.corr()</code>	Correlation matrix
<code>df.corr()['target'].sort_values()</code>	Features most correlated with target
<code>df.select_dtypes(include=['number'])</code>	Select only numeric columns
<code>np.log1p(df['col'])</code>	Log transform — good for skewed features
<code>pd.cut(df['col'], bins=5)</code>	Bin continuous column into 5 ranges
<code>pd.qcut(df['col'], q=4)</code>	Bin by quantile (equal size buckets)
<code>df['col'].rolling(7).mean()</code>	7-period rolling mean (time series)
<code>df['col'].shift(1)</code>	Lag feature — previous row's value

Tip: Always run `df.info()` and `df.describe()` first on any new dataset. Then `df.isnull().sum()` to check missing values.